Objetivos*:*

- *Comprender la interacción **cliente–servidor** en una aplicación web.*

- *Implementar el flujo básico de una **solicitud HTTP** en Laravel.*

- *Generar **respuestas dinámicas** en función de datos enviados por el usuario.*

Conceptos:

Cliente – Servidor: Es un modelo de arquitectura donde una aplicación se divide en dos partes:

- **Cliente:** aplicación que realiza solicitudes (generalmente el navegador web).
- **Servidor:** sistema que procesa esas solicitudes y devuelve una respuesta.

En aplicaciones web, el cliente solicita recursos mediante Internet y el servidor genera dinámicamente el contenido que el usuario visualiza.

Petición HTTP: es el mensaje que envía el cliente al servidor para solicitar o enviar información. Los métodos más utilizados son:

- **GET:** solicita información al servidor (por ejemplo, abrir una página).
- **POST:** envía datos al servidor para ser procesados (por ejemplo, enviar un formulario).

Cada interacción web ocurre mediante una petición y una respuesta HTTP.

Controlador: es un componente del backend encargado de:

- recibir una solicitud del cliente,
- procesar la información,
- decidir qué respuesta devolver.

En Laravel, los controladores permiten organizar la lógica de la aplicación separándola de las vistas y las rutas.

Respuesta dinámica: Es una página web cuyo contenido se genera en el servidor utilizando datos o lógica de programación.

A diferencia de una página estática, el contenido puede cambiar según:

- los datos enviados por el usuario
- el estado del sistema
- reglas definidas en el backend

Ejemplo: mostrar un mensaje personalizado con el nombre ingresado en un formulario.

En esta actividad continuaremos trabajando en el proyecto Laravel creado en la guía práctica 1 y modificados en las guías 2 y 3.

Actividades Prácticas

Actividad 1: Procesamiento de un formulario web

En esta actividad aprenderemos cómo una aplicación web procesa información enviada desde el navegador.

Crear un formulario de contacto simple que permitirá al usuario ingresar:

- Nombre
- Correo electrónico
- Mensaje

Es similar a las actividades realizadas en las guías practicas 2 y 3 😉

1. Crear el archivo de la vista **contacto.blade.php** y en el crear un formulario web utilizando HTML y Bootstrap.
   Aquí tienes la guía de Bootstrap con ejemplos de formularios:
   https://getbootstrap.com/docs/5.3/forms/overview/#overview

   ¡Importante!
   En la etiqueta **form** debes incluir la ruta a la que se enviará el formulario y el método HTTP que utilizará para el envío. Para eso se utiliza **action** y **method** como se muestra a continuación:

   ```
   <form action="{{ url('/contacto') }}" method="POST">
   ```

2. Crear el archivo de la vista **exito.blade.php**. Esta vista debe mostrar el mensaje **"Recibimos tu mensaje. Un asesor comercial se comunicará contigo a la brevedad. ¡Muchas gracias!"**
   Esta vista se mostrará como respuesta del servidor luego de completar y enviar el formulario de contacto.
3. Crear el controlador **ContactoController**:
   Abre la consola de comandos (cmd o powershell en Windows) en la ubicación de tu proyecto y ejecuta el comando **php artisan make:controller NombreDelControlador** para crear un nuevo controlador. A continuación tienes un ejemplo para crear el controlador **ContactoController**:

   ```
   php artisan make:controller ContactoController
   ```

Ahora, otra vez en el IDE de desarrollo, veremos un nuevo archivo de controlador en la ruta **app/Http/Controllers** llamado **ContactoController.** Debemos crear el método **"procesar"** para retornar la vista de éxito cuando se envía un formulario desde el navegador web.

```
public function procesar(Request $request)
   {
       return view('exito');
   }
```

4. Crear las rutas para cargar la vista del formulario (GET) y permitir en envío del formulario (POST).
   En el archivo de rutas web.php trabajado en las guias prácticas anteriores agregar al final las 2 nuevas rutas:

```
Route::get('/contacto', function () {
   return view('contacto');
});
```

   Retorna la vista **contacto** cuando se realiza una **petición GET** a **/contacto**

```
Route::post('/contacto', [ContactoController::class, 'procesar']);
```

   Cuando se realiza una **petición POST** a /contacto se llama al método **'procesar'** del controlador **ContactoController** (creado en el paso anterior)

5. Probar la aplicación
   Abrir en navegador y acceder a la URL del proyecto + /contacto:
   http://primer_proyecto_laravel.test/contacto

<u>Actividad 2</u>: Respuesta personalizada

Modificar la aplicación para que el servidor utilice los datos enviados por el usuario.

1. Modificar el controlador para capturar los datos enviados desde el formulario y pasarlos a la respuesta (vista 'exito')
   En el método procesar debemos capturar los datos del formulario:

```
$nombre = $request->input('nombre');
$email = $request->input('email');
$mensaje = $request->input(mensaje);
```

   Retornaremos los valores obtenidos en $nombre y $email para crear una experiencia de usuario personalizada en la vista 'éxito'

```
return view('exito', [
    'nombre' => $nombre,
    'email' => $email
]);
```

2. Debemos modificar también la vista exito.blade.php para recibir estos parámetros y mostrarlos al usuario.
   Para poder utilizar los parámetros recibidos solo debemos utilizar el nombre de la variable dentro de doble llave en el código HTML.
   Por ejemplo, para mostrar al usuario el mensaje: "Hola **José**, que bueno recibir tu mensaje. Un miembro del equipo de ventas se pondrá en contacto con vos al correo: **jose@gmail.com** ¡Muchas gracias!":

```
<p class="lead">
    Hola <strong>{{ $nombre }}</strong>, qué bueno recibir tu mensaje. Un miembro
del equipo de ventas se pondrá en contacto con vos al correo: <strong>{{ $email
}}</strong> ¡Muchas gracias!
</p>
```

3. Vuelve a probar en el navegador
   Completa y envía un formulario de contacto para ver la respuesta personalizada.
   http://primer_proyecto_laravel.test/contacto

Referencias:

- Documentación oficial de Bootstrap
- Controladores de Laravel
- Responses en Laravel